

# **RAPPORT TECHNIQUE**

## **Mise en œuvre d'un processus ETL répondant au besoin d'une solution I.A**

ObRail

MSPR, Bloc E6.1

EPSI Montpellier | DEVIA B3 2025-2026

MASUTTI Lucas, MATTERA Lucas, SHIN Dounghwi, BALOKA TELMAN Esdras Emmanuel

## Sommaire

|                                           |    |
|-------------------------------------------|----|
| 1. Contexte et périmètre                  | 3  |
| 2. Sources de données                     | 4  |
| 3. Architecture technique                 | 5  |
| 4. Processus ETL                          | 6  |
| 5. Modèle de données                      | 8  |
| 6. API REST                               | 9  |
| 7. Tableau de bord                        | 10 |
| 8. Qualité des données et conformité RGPD | 11 |
| 9. Limites et perspectives                | 12 |

# 1. Contexte et périmètre

## 1.1 Commanditaire

ObRail Europe est un observatoire indépendant, spécialisé dans l'analyse des flux ferroviaires européens. Sa mission est d'évaluer l'empreinte carbone des déplacements longue distance et produire des études comparatives sur les réseaux ferroviaires et aériens à destination des décideurs politiques et des opérateurs.

## 1.2 Problématique

Les données ferroviaires européennes sont publiées par énormément d'acteurs distincts, dans des formats différents et sans référentiel commun. L'objectif de ce projet est de centraliser ces données via un processus ETL automatisé, de les homogénéiser, puis de les exposer via une API REST.

## 1.3 Périmètre

Zone géographique couverte : Europe (choix des pays à inclure dans les variables). Les données aériennes sont intégrées à des fins de comparaison carbone. Livrables : modèle de données, scripts ETL, base PostgreSQL, API FastAPI, tableau de bord HTML.

# 2. Sources de données

## 2.1 Format GTFS

Le format GTFS (General Transit Feed Specification) est le standard retenu pour la collecte des données ferroviaires. Cinq fichiers contenus dans les GTFS sont exploités : *routes.txt*, *trips.txt*, *stops.txt*, *stop\_times.txt*, *agency.txt*. Ce format est disponible publiquement pour la majorité des opérateurs européens, sans authentification.

## 2.2 Sources mobilisées

Les flux sont récupérés dynamiquement depuis le catalogue Mobility Database (fichier *feeds\_v2.csv*), filtré sur les pays cibles et les *route\_type* ferroviaires (2, 100–117). Les sources statiques suivantes sont définies en complément :

- SNCF Open Data : horaires TGV, Intercités, TER, RER
- GTFS.DE : réseau ferroviaire allemand (Fernverkehr + Regionalverkehr)
- OpenMobility / ATOC : réseau ferroviaire britannique
- HSL (Helsinki Region Transport) : trains de banlieue et métro d'Helsinki
- PID (Prague Integrated Transport) : trains régionaux de Prague
- Eurocontrol OPDI : données de vols (fichiers Parquet mensuels), croisées avec le référentiel OurAirports

Un mécanisme d'exclusion (*ignorelist*) évite de retenter les URLs défaillantes entre deux exécutions.

## 3. Architecture technique

### 3.1 Vue d'ensemble

L'architecture repose sur trois composants : un processus ETL Python/Spark, une base PostgreSQL, et une API REST FastAPI. Un tableau de bord HTML statique complète l'ensemble.

### 3.2 Stack technique

| Composant                 | Technologie                           | Justification                                         |
|---------------------------|---------------------------------------|-------------------------------------------------------|
| ETL                       | Python 3.10+, PySpark (local[*])      | Traitement distribué, gestion de volumes > 10M lignes |
| Base de données           | PostgreSQL (schéma obrail_transport)  | Robustesse, indexation avancée, support JSON          |
| API REST                  | FastAPI + psycpg2 + Pydantic          | Performances, validation automatique, doc OpenAPI     |
| Téléchargement            | requests (streaming)                  | Gestion des timeouts, retry, fichiers volumineux      |
| Fichiers XLSX             | pandas + openpyxl                     | Lecture des flux GTFS au format Excel multi-onglets   |
| Calcul distances          | Haversine (expressions Spark natives) | Calcul GPS sans UDF Python, performances optimales    |
| Émissions CO <sup>2</sup> | Coefficients ADEME                    | Référence officielle française                        |

### 3.3 Configuration

Tout le processus est piloté par variables d'environnement. Aucune valeur n'est codée en dur dans le code source.

- *INPUT\_FILES / INPUT\_FILE\_LIST* : URLs ou chemins locaux des flux à traiter
- *PGHOST, PGPORT, PGDATABASE, PGUSER, PGPASSWORD* : connexion PostgreSQL
- *GTFS\_COUNTRY\_CODES* : codes ISO des pays à inclure dans la collecte catalogue
- *MIN\_DISTANCE\_KM* : seuil de distance minimum (défaut : 100 km)
- *DISTANCE\_FACTOR\_TRAIN* : facteur de correction Haversine pour le ferroviaire (défaut : 1.3)
- *FLIGHTS\_ENABLE* : activation du traitement des données aériennes
- *SPARK\_DRIVER\_MEMORY, SPARK\_EXECUTOR\_MEMORY* : dimensionnement mémoire Spark

## 4. Processus ETL

### 4.1 Extraction

La liste des sources est construite en fusionnant trois canaux : *INPUT\_FILES*, découverte automatique depuis Mobility Database, et sources statiques. Le résultat est dédoublé avant traitement.

Chaque source est téléchargée en streaming (2 tentatives par défaut). Les archives ZIP sont extraites dans un répertoire temporaire dédié à chaque exécution, nettoyé via un bloc finally. Seuls les cinq fichiers utiles sont extraits (*routes.txt*, *trips.txt*, *stop\_times.txt*, *stops.txt*, *agency.txt*).

### 4.2 Transformation

#### Filtrage des types de transport

Seuls les *route\_type* ferroviaires (2, 100–117) et aériens (1100–1107) sont conservés. Cela couvre : trains longue distance, régionaux, grande vitesse, trains de nuit, et vols commerciaux.

#### Détection des trains de nuit

Un trajet est qualifié train de nuit si :

- son *route\_type* GTFS vaut 105,
- ou son nom de ligne contient l'un des termes : 'night', 'nuit', 'noct', 'overnight', 'sleeper', 'nightjet', 'couchette'.

#### Calcul des distances

La distance est calculée via la formule de Haversine, implémentée en expressions Spark natives (pas d'UDF Python, pour préserver les performances). Un facteur de correction de x1.3 est appliqué pour les trains, afin de compenser le tracé non rectiligne des voies et simuler un vrai trajet. Les trajets inférieurs à *MIN\_DISTANCE\_KM* sont exclus.

#### Calcul des émissions CO<sup>2</sup>

Les émissions sont estimées à partir des coefficients ADEME suivants, paramétrables via variables d'environnement :

- TGV : 0,0029 kg CO<sup>2</sup>/km
- Intercités : 0,009 kg CO<sup>2</sup>/km
- RER : 0,0277 kg CO<sup>2</sup>/km
- Trains standard (défaut) : 0,009 kg CO<sup>2</sup>/km
- Avion : 0,1008 kg CO<sup>2</sup>/km

Le type de train est déterminé en croisant le *route\_type* GTFS avec le nom de la ligne (détection de 'TGV', 'RER', 'Intercité', etc.).

#### Résolution des arrêts parents

La fonction *\_resolve\_parent\_stops* remonte vers l'arrêt parent (champ *parent\_station*) pour hériter de son nom et de ses coordonnées GPS. Cela évite les doublons de quais pour une même gare.

#### Gestion des pays et validation géographique

Le code pays est récupéré dans l'ordre de priorité suivant : champ *stop\_country* du flux GTFS, déduction depuis le catalogue Mobility Database, valeur par défaut via *DEFAULT\_COUNTRY\_CODE*. La validation par bounding box (*GTFS\_VALIDATE\_COUNTRY\_BBOX=1*) invalide les arrêts dont les coordonnées GPS ne correspondent pas au pays déclaré.

### 4.3 Déduplication

Deux niveaux de déduplication sont appliqués :

- Stations : clé composite (nom normalisé + latitude arrondie à 4 décimales + longitude + pays)
- Trajets : normalisation de la direction (pair\_a / pair\_b, ordonné par ID croissant). Un trajet A→B et son inverse B→A sont traités comme une seule liaison.

### 4.4 Chargement

L'écriture en base se fait via JDBC Spark, directement depuis les DataFrames vers PostgreSQL. En cas d'échec, une seconde tentative est lancée avec moins de partitions et des lots plus petits. Les tables sont tronquées avant chaque chargement complet (*ETL\_TRUNCATE=1* par défaut). L'option *ETL\_SNAPSHOT\_WRITE* permet de sauvegarder les DataFrames en Parquet avant l'écriture.

## 5. Modèle de données

### 5.1 Schéma

Le schéma PostgreSQL obrail\_transport contient trois tables : *vehicule*, *station*, *trajet*. Le modèle en étoile simplifié a été retenu pour sa lisibilité et sa facilité de maintenance.

### 5.2 Table *vehicule*

| Colonne        | Type         | Description                                       |
|----------------|--------------|---------------------------------------------------|
| vehicule_id    | INTEGER (PK) | Identifiant séquentiel auto-généré                |
| type_transport | VARCHAR(16)  | Catégorie : 'train' ou 'avion'                    |
| specificite    | VARCHAR(256) | Désignation détaillée (TGV, ICE, Night...)        |
| train_type     | INTEGER      | Code GTFS du type de route (ex : 102 = Intercité) |

### 5.3 Table *station*

| Colonne      | Type         | Description                               |
|--------------|--------------|-------------------------------------------|
| station_id   | INTEGER (PK) | Identifiant séquentiel                    |
| stop_id      | VARCHAR(128) | Identifiant GTFS original (traçabilité)   |
| station_name | VARCHAR(256) | Nom normalisé de la gare ou de l'aéroport |

| Colonne   | Type             | Description                              |
|-----------|------------------|------------------------------------------|
| latitude  | DOUBLE PRECISION | Latitude WGS84 (arrondie à 4 décimales)  |
| longitude | DOUBLE PRECISION | Longitude WGS84 (arrondie à 4 décimales) |
| pays      | VARCHAR(8)       | Code ISO 3166-1 alpha-2 du pays          |

## 5.4 Table *trajet*

| Colonne              | Type             | Description                                          |
|----------------------|------------------|------------------------------------------------------|
| trajet_id            | BIGINT (PK)      | Identifiant séquentiel                               |
| vehicule_id          | INTEGER (FK)     | Référence vers vehicule                              |
| is_night             | BOOLEAN          | Indicateur train de nuit                             |
| departure_station_id | INTEGER (FK)     | Station de départ                                    |
| arrival_station_id   | INTEGER (FK)     | Station d'arrivée                                    |
| distance_km          | DOUBLE PRECISION | Distance estimée (Haversine × facteur)               |
| co2_kg               | DOUBLE PRECISION | Émissions CO <sup>2</sup> estimées en kg             |
| departure_time       | VARCHAR(16)      | Heure de départ GTFS (HH:MM:SS, HH peut dépasser 23) |
| arrival_time         | VARCHAR(16)      | Heure d'arrivée GTFS (HH:MM:SS)                      |
| agency_timezone      | VARCHAR(64)      | Fuseau horaire de l'opérateur                        |

Index définis sur *vehicule\_id*, *departure\_station\_id* et *arrival\_station\_id* pour optimiser les jointures.

## 6. API REST

### 6.1 Technologies

L'API est développée avec FastAPI (Python). La connexion à PostgreSQL est gérée par psycopg2 via le système de dépendances FastAPI (Depends). Les modèles de réponse sont définis avec Pydantic. La documentation interactive OpenAPI/Swagger est générée automatiquement.

### 6.2 Endpoints

| Route       | Description                |
|-------------|----------------------------|
| GET /       | Tableau de bord HTML       |
| GET /health | Statut de l'API et version |

| Route               | Description                                                                |
|---------------------|----------------------------------------------------------------------------|
| GET /stats          | Statistiques globales (trajets, véhicules, stations, trains de nuit)       |
| GET /stats/coverage | Répartition des trajets par pays (jour / nuit / transfrontalier)           |
| GET /stats/quality  | Indicateurs de complétude par champ                                        |
| GET /countries      | Liste des pays couverts avec code ISO                                      |
| GET /operators      | Liste des opérateurs, filtrable par type_transport                         |
| GET /vehicules      | Types de véhicules avec filtres multiples                                  |
| GET /stations       | Stations avec filtres (pays, nom, type de transport)                       |
| GET /trajets        | Trajets avec filtres multiples (nuit, pays, opérateur, distance...)        |
| GET /trips          | Trajets enrichis avec détection transfrontalière et conversion horaire CET |

### 6.3 Paramètres de filtrage

Les routes de données partagent un ensemble de paramètres cohérents. Sur */trajets* : *type\_transport*, *specificite*, *train\_type*, *is\_night*, *pays* (ISO), *departure\_station*, *arrival\_station* (recherche partielle insensible à la casse), *min\_distance\_km*, *max\_distance\_km*. Pagination via *limit* (max 20 000) et *offset*.

La route */trips* accepte un paramètre *service\_date* qui, combiné à *agency\_timezone*, convertit les horaires GTFS en heure CET (Europe/Paris).

### 6.4 Sécurité

Toutes les requêtes SQL utilisent des paramètres préparés (%s avec psycopg2). Les paramètres textuels sont soumis à des contraintes de longueur via Query() FastAPI. Les identifiants de connexion sont chargés depuis des variables d'environnement.

## 7. Tableau de bord

Le tableau de bord est une page HTML statique servie via la route *GET /*. Aucune dépendance externe : les visualisations sont générées côté client en JavaScript, en interrogeant l'API au chargement de la page.

Indicateurs affichés :

- Statistiques globales : nombre de trajets, véhicules, stations, proportion de trains de nuit
- Répartition jour / nuit
- Couverture par pays
- Taux de complétude par champ (distance, CO<sup>2</sup>, pays, coordonnées GPS)
- Listes des opérateurs et stations filtrables par pays



## 8. Qualité des données et conformité RGPD

### 8.1 Contrôles qualité

En amont : exclusion des arrêts avec coordonnées GPS nulles, hors plage  $[-90,90] \times [-180,180]$  ou égales à zéro. Validation optionnelle par bounding box (`GTFS_VALIDATE_COUNTRY_BBOX=1`) pour détecter les incohérences pays / coordonnées.

En sortie : la route `/stats/quality` expose les indicateurs de complétude suivants : taux de trajets sans distance, sans CO<sup>2</sup>, sans code pays, sans coordonnées GPS.

### 8.2 Traçabilité

Chaque exécution est journalisée dans `transport_etl.log`. L'option `ETL_SNAPSHOT_WRITE` permet de conserver un instantané des données transformées en Parquet avant chargement.

### 8.3 Conformité RGPD

Les données traitées sont exclusivement publiques : horaires ferroviaires, géolocalisation des gares, types de véhicules. Aucune donnée personnelle n'est collectée, stockée ou exposée. Mesures appliquées :

- Sources documentées avec leurs licences
- Fichiers temporaires supprimés après chaque exécution (`shutil.rmtree`)
- Identifiants de connexion chargés depuis des variables d'environnement

## 9. Limites et perspectives

### 9.1 Limites actuelles

Couverture géographique : certains pays d'Europe de l'Est publient peu de flux GTFS ferroviaires accessibles. La Suisse a été exclue du périmètre (flux contenant des milliers de trajets très courts et similaires). Les trains de nuit internationaux ne sont pas toujours disponibles en accès libre.

Déduplication directionnelle : la normalisation A→B supprime l'information sur le sens de circulation, ce qui peut être limitant pour certaines analyses (lignes à sens unique, liaisons asymétriques).

Performance : Spark en mode local est gourmand en ressources. Le téléchargement séquentiel de plusieurs dizaines de flux peut prendre plusieurs heures. Des erreurs `OutOfMemoryError` peuvent survenir sur les flux volumineux (`stop_times.txt` de la SNCF ou de la Deutsche Bahn pouvant dépasser plusieurs dizaines de millions de lignes). Le script réduit le nombre de partitions avant l'écriture JDBC en cas d'échec, mais cela reste insuffisant sur des machines disposant de moins de 16 Go de RAM.

### 9.2 Perspectives d'amélioration

Sources de données : intégration du format HAFAS (Deutsche Bahn, opérateurs semi-ouverts) pour une meilleure couverture grande vitesse et trains de nuit. Croisement avec les données Eurostat sur les flux de passagers.

Performance : parallélisation des téléchargements (`asyncio` ou `ThreadPoolExecutor`). Remplacement du truncate/reload complet par un système de mise à jour en delta. Filtrage

de *stop\_times.txt* en amont pour ne conserver que les arrêts de départ et d'arrivée avant chargement dans Spark.

API : mise en place d'une authentification (tokens JWT ou clés d'API) pour tracer l'utilisation par partenaire et gérer des quotas.

## Conclusion

Ce projet met en place un processus ETL complet et automatisé, capable d'ingérer des données ferroviaires depuis plusieurs dizaines de sources européennes, de les homogénéiser et de les charger dans un entrepôt relationnel structuré. L'API FastAPI expose ce référentiel de façon flexible et documentée. Le tableau de bord HTML offre une interface de pilotage accessible à tous profils. Le système est conçu pour être reproductible, paramétrable et évolutif, ce qui prépare les étapes suivantes du projet d'ObRail : développement de modèles prédictifs et mise en production d'applications métier.